

Autonomous Object Retrieval Robot with YOLOv11 Detection

EIE3360 INTEGRATED PROJECT – PROJECT REPORT

Group member:

LI Ming Chun Simon (25017659D)

Hon Ching Kwong (25021497D)

ZHONG Jiawei (21098707D)

Group 17 Lab001

Contents

1. Introduction	2
2. Background and Objectives	3
3. Methodology and Description	5
3.1 Hardware Components	5
3.2 Software Components	5
3.3 Object Detection Process	6
3.4 Software–Hardware Integration	6
4. Milestone Achievement	7
5. Implementation and System Integration	8
5.1 Object Detection Configuration	8
5.2 Mechanical Structure Reliability and Limitations	9
5.3 Additional Features Implemented	10
5.4 Challenges Encountered and Solutions	10
5.5 Testing Procedures	11
5.6 Strategies and Techniques for Learning to Learn (L2L)	12
5.7 Team Collaboration	13
6. Results and Evaluation	14
6.1 Outcomes of the Project	14
6.2 Evaluation of Success Against Objectives	14
6.3 Areas for Improvement	15
7. Conclusion	16
8. References and Appendix	17

1. Introduction

This project presents an autonomous visual-guided robot system designed to perform a practical sorting and handling task in a confined indoor environment. The robot needs to identify four different beverage items (Coca-Cola, Sprite, Vita Lemon Tea, and Vitasoy Soybean Milk), determine the correct placement area for each item (green balls correspond to Coca-Cola and Sprite, red balls correspond to Vita Lemon Tea and Vitasoy Soybean Milk), approach the items, use a servo-driven gripper to grasp the items, transport them to the area near the corresponding color balls, and then release the items - the entire process requires no form of remote control or manual intervention. The entire system runs on the NVIDIA Jetson Orin embedded platform, which provides high computational throughput for deep learning inference while maintaining the low power consumption required for wireless robot operation.

The main objective is to demonstrate a robust end-to-end integration between modern computer vision (YOLOv11 object detection), classical control theory (PID controller), and task orchestration based on finite state machines (FSM) on resource-constrained edge hardware. In addition to simple detection, the system must also handle real-world challenges such as target loss, occlusion, multiple objects appearing in the same frame, and the need to avoid grasping items that are too close to the placement area.

This implementation features several key innovations:

1. **Non-blocking search pulse mechanism** - When the robot loses the target, many simple implementations use blocking `time.sleep()` calls, drastically reducing the frame rate (typically down to 2-3 FPS). Our implementation uses timestamp-based state variables (`search_turn_end_time`, `search_pause_end_time`) instead of blocking sleep. The main loop checks these timestamps in each iteration, allowing the robot to make a short duration turn for a short period and then stop, never blocking the camera capture or detection process. Even during the search, high-frequency visual feedback can be maintained.
2. **Adaptive Anti-Grab Mask Strategy** - To prevent the robot from grasping items that are too close to the dropped ball, the visual system will dynamically construct an elliptical anti-pulling mask around each detected ball, project it onto

the ground plane and expand it. If a sample point at the lower part of a beverage item intersects with this mask, it is considered that the item cannot be grasped.

3. **Post-grab category-based turning** - After picking up an item, the robot will query the category of the carried item: Coca-Cola and Sprite trigger a left turn, while Vita Lemon Tea and Vitasoy Soybean Milk trigger a right turn. This ensures that the robot always faces the correct side of the field before searching for the corresponding-colored balls.
4. **Motor command frequency limitation and de-duplication** – The actuator ensures that there is at least a 50-millisecond interval between consecutive motor commands and skips nearly identical commands, thereby reducing serial bus congestion and unnecessary motor jitter.

The resulting system demonstrates responsive, low-latency autonomous sorting, confirming the feasibility of embedded AI for practical service robotics.

2. Background and Objectives

With the emergence of powerful and energy-efficient platforms such as NVIDIA Jetson series, embedded AI has achieved significant maturity. These platforms enable complex deep learning models to run directly on mobile robots without relying on cloud - this is crucial for real-time applications where network latency or privacy issues do not allow cloud offloading. The Jetson Orin provides sufficient AI performance to run the YOLOv11 model at real-time frame rates (23-42 FPS), while handling camera capture, motor control, and state machine logic.

However, deployment remains challenging. The main difficulties include:

- **Resource competition** - GPUs, CPUs and memory are shared among neural network inference and real-time tasks. Inefficient scheduling can lead to peak detection delays or frame loss.
- **Deterministic timing** - Motor control requires regular updates (20-50 Hz). Unpredictable blocking from visual processing can lead to overshoot or oscillation.
- **Target loss recovery** – When the robot turns or the target is blocked, the detector may not output bounding boxes. The control system must gracefully degrade to

search behavior and then resume tracking.

- **Spatial reasoning in the no-grab zone** – The robot must avoid grasping items positioned too close to the balls or their supporting cans, as this could disturb the ball and disrupt the color-coded area association.

In this context, this project has set the following specific goals:

1. **Real-time multi-class detection** - On the Jetson Orin, using the YOLOv11 model exported as TensorRT, four beverage categories and two ball types can be reliably detected at a frame rate of at least 23 FPS. The confidence threshold is set to 0.6, achieving a balance between accuracy and recall.
2. **Autonomous target approach based on PID control** - Implement a PID controller, with the horizontal pixel error ($\text{center_x} - \text{target_x}$) as the input, and output the motor speed correction value. The PID gains ($K_p = 9.0$, $K_i = 0.05$, $K_d = 3.5$) are tuned through experiments to provide rapid and stable tracking, and an integral anti-saturation mechanism is adopted.
3. **Finite state machine for task orchestration** - Implement and test a five-state FSM: $\text{startup_forward} \rightarrow \text{search_item} \rightarrow \text{grab} \rightarrow \text{search_drop} \rightarrow \text{release} \rightarrow \text{return to search_item}$. Each state has clear entry/exit conditions and timeouts.
4. **Avoidance of capture zones** – Utilizing a ground projection model, dynamically calculate the elliptical prohibited capture areas around each detected ball. Cross-frame persistent masks are employed to handle the temporary loss of small ball detections.
5. **Robust search without causing frame rate collapse** - Replace all blocking `time.sleep()` calls in the search logic with non-blocking delays based on timestamps, ensuring that the main loop can continue to read the camera and run the detection at full speed (23-42 FPS) even without visible targets.

Achieving these goals will result in a robot capable of autonomous and continuous sorting, demonstrating the practical value of AI in the field of logistics and service robots.

3. Methodology and Description

3.1 Hardware Components

The robot's hardware platform consists of:

- **Embedded computer:** NVIDIA Jetson Orin (8GB RAM), Ubuntu 20.04 – runs all software.
- **Camera:** C70 Metal Shell Wide FOV Camera – provides visual input.
- **Motor driver & motors:** Dual DC motors with encoders, controlled via lib3360 – differential drive.
- **Servo gripper:** Two-position servo (home ~1400 μ s, grab/release ~1000 μ s) – picks up and releases items.
- **Chassis:** Four-wheel differential-drive platform with front-mounted gripper.
- **Power:** Rechargeable Li-ion battery pack (12V).
Communication with motor driver and servo is via a USB-to-serial adapter; the camera connects via USB 3.0 for sufficient bandwidth.

3.2 Software Components

The software is structured into cohesive modules:

- **RobotConfig (dataclass)** – Centralizes all tunable parameters: PID gains, speeds, timeouts, thresholds, and class lists. Enables easy recalibration without code changes.
- **RobotActuator** – Wraps low-level motor() and servo() functions. Implements command deduplication (skips identical commands), rate limiting (max 20 Hz), and high-level methods: forward(), backward(), turn_left(), turn_right(), drive_with_pid(), set_servo_home(), set_servo_grab_or_release().
- **VisionSystem** – Loads a TensorRT-engine YOLO model. The detect() method crops and resizes input to 416×416, runs inference (conf=0.6, device=0), and maps bounding boxes back to original coordinates. build_forbidden_mask() draws ellipses around detected balls, projects them onto the ground plane, dilates the mask, and persists it for up to 15 frames.
- **PIDState & compute_pid()** – Implements standard PID with integral anti-windup. Correction is clipped to $\pm$ pid_max_correction and rate-limited to prevent jerky movements.

- **RuntimeState** – Holds current state, timestamps, target locking information, lost_frames counter, and non-blocking search timers (search_turn_end_time, search_pause_end_time).
- **RobotController** – Main orchestrator. The run() method reads frames, calls appropriate state handlers, periodically runs detection, selects targets, and controls actuators.

3.3 Object Detection Process

The object detection subsystem uses a custom-trained YOLOv11 model deployed on the Jetson Orin with TensorRT optimization. The detection pipeline captures frames from the USB camera, crops and resizes them to 416×416 pixels, and runs inference with a confidence threshold of 0.6. Bounding box coordinates are mapped back to the original frame resolution for use in navigation logic.

To prevent the robot from targeting objects positioned behind or too close to the balls, a forbidden mask is generated. For each detected ball, an elliptical region is projected onto the ground plane beneath it, dilated, and persisted in memory for up to 15 frames. This prevents invalid target selection during brief occlusions.

Detailed dataset composition, training parameters, and performance metrics are provided in Section 5.1.

3.4 Software–Hardware Integration

Integration is achieved through a carefully designed main loop respecting real-time constraints:

- **Non-blocking design** – No time.sleep() blocks the main loop except during short, predictable grab/release sequences. In search states, _handle_search_pulse() uses timestamp comparisons to generate turn pulses without stopping the loop — critical for maintaining high FPS during target loss.
- **PID loop frequency** – PID updates on every frame where a target is detected. The correction is applied immediately via drive_with_pid(). With detection at 23-42 FPS, motor commands update at roughly detection rate — sufficient for smooth differential control.

- **State machine transitions** – Triggered by timeouts (e.g., 9-second startup forward), object width thresholds (e.g., width > 380 triggers grab), or completion of fixed-duration actions.
- **Forbidden mask memory** – When balls are temporarily undetected, `build_forbidden_mask()` returns the last valid mask for up to 15 frames, preventing sudden invalid target selection during brief occlusion.
- **Servo coordination** – Servo actuation occurs only at specific moments: gripper closes after forward movement in grab, opens after blind-release forward in release. Between actions, the servo remains in home (open) position.

The result is a tightly coupled yet resource-efficient system where vision, decision, and action operate without starving each other — all on a single Jetson Orin.

4. Milestone Achievement

By the end of the project, most of the milestones we set in the proposal had been completed successfully. These milestones were important because they showed our progress from individual functions, such as object detection and movement control, to a fully integrated autonomous robot system.

First, we successfully completed both **Game 1** and **Game 2**, which means the robot was able to meet the main task requirements. We also built our own dataset for the project. The final dataset contained **2,564 annotations** across nine classes, including different beverage items and the red and green marker balls. This helped improve the detection accuracy in the real competition environment.

Another important milestone was that the robot could complete the full process of **recognition, approaching, grasping, transporting, and releasing**. This showed that our system was no longer just doing object detection but was able to perform the whole task in a closed loop.

On the hardware side, we designed and tested **two versions of a 3D-printed gripper**, and the final version was used in the demonstration. This was a key improvement because the original gripper was not reliable enough for grabbing paper-pack drinks. After the redesign, the robot became much more stable when handling both cartons

and cans.

Overall, our project achieved the main milestones: completing both games, building a task-specific dataset, deploying the detection model on the embedded platform, finishing the full autonomous pipeline, integrating the final gripper design. In the final demonstration, the robot showed a **high task completion rate** and was able to complete the required workflow in a stable manner under the real competition setup.

5. Implementation and System Integration

5.1 Object Detection Configuration

The object detection subsystem used YOLOv11 deployed on NVIDIA Jetson Orin Nano with TensorRT optimization. The final implementation parameters are as follows.

Number of Objects to Detect:

Six classes: Coca-Cola, Vita Lemon Tea, Vitasoy Soybean Milk, sprite, green ball, and red ball. Classes Coca-Cola black, cream soda, and fanta were annotated but excluded from active detection as they were either distractors or fell outside the camera's near-field view during approach.

Number of Images in Dataset:

The dataset contained 1,652 images with 2,564 annotations. Class distribution is shown in Table 1.

Table 1: Dataset Class Distribution

Class Name	Annotation Count
Coca-Cola	233
Coca-Cola black	339
cream soda	303
fanta	210
green ball	360
red ball	333
sprite	241
Vita Lemon Tea	337

Vitasoy Soybean Milk	208
Total	2564

Number of Training Epochs:

100 epochs, determined by monitoring loss convergence to avoid overfitting.

Confidence Level Threshold:

0.6 as defined in VisionSystem.detect(). This balanced sensitivity against false positives while maintaining reliable detection during motion.

Impact on System Performance:

With 416×416 model input size and TensorRT .engine format, inference ran at **23-42 FPS**, sufficient for real-time PID-based navigation. Detection confidence routinely exceeded 0.9 during successful runs.

5.2 Mechanical Structure Reliability and Limitations

Reliability: The four-wheel differential-drive chassis with Jetson Orin Nano provided a stable platform after modifications. The custom gripper enabled reliable gripping of both can and paper-pack formats.

Limitations and Mitigation:

- **Wheel-to-chassis friction:** Stock configuration caused inconsistent straight-line tracking. Mitigated by adding four metal standoffs to raise the upper plate and inserting Blu Tack into axle housings to offset wheels outward, reducing contact with chassis edges.
- **Stock gripper incompatibility:** Original gripper (≈ 5.1 cm) could not grip paper-pack drinks. A custom 3D-printed gripper of ≈ 8 cm length with optimized jaw width was designed, enabling secure grip on both cylindrical cans and rectangular paper-packs.
- **Lack of proximity sensing:** Monocular vision created a near-field blind spot. Software implemented a blind-drop mechanism using object width threshold (drop_threshold = 180 pixels) to trigger timed forward-and-release without continuous visual lock.

5.3 Additional Features Implemented

Hardware:

- **Custom 3D-Printed Gripper:** Extended length (≈ 8 cm) and adjusted jaw geometry enabled reliable gripping of both can types. **Anti-slip pads** on inner surfaces increased friction and reduced slippage during transport.
- **Chassis Clearance Modifications:** Additional standoffs and axle offset spacers reduced wheel binding and improved straight-line navigation stability.

Software:

- **State machine:** Operation is divided into five states (startup_forward, search_item, grab, search_drop, release), making the logic clearer and easier to debug.
- **PID-based steering correction:** Replaced simple left/right region checks with continuous wheel speed adjustment based on target position error, enabling smoother and more precise approach.
- **Target locking and switching control:** Maintains focus on current target and only switches when a clearly better candidate appears, preventing unstable target hopping in multi-object scenes.
- **Forbidden-zone logic:** Generates restricted areas near marker-ball regions to avoid selecting unsafe targets and reduce risk of disturbing drop-zone markers.
- **Lost-target recovery:** Performs short search routines when target is temporarily lost, using non-blocking search pulses to maintain system responsiveness without long delays.
- **Drop-zone mapping:** Automatically selects the correct colored marker based on the grabbed item and executes the appropriate release sequence, enabling full sorting functionality beyond basic detection and movement.

5.4 Challenges Encountered and Solutions

During the implementation, we met several problems with both software and hardware, and most features in our final code were added to solve these problems.

One major problem was that **the original control logic was too simple** for a complete sorting task. The basic code could handle only a limited situation, but when we

needed the robot to recognize different drinks, remember the current object, search for the correct marker, and complete release actions, the logic became difficult to manage. We addressed this by restructuring the program as a finite state machine, which improved organization and debugging.

Another challenge was **unstable movement during target tracking**. At first, even when the robot could detect the object, it sometimes moved too roughly and could not stay well aligned with the target. To solve this, we added PID steering correction to enable smoother direction adjustments based on target position error.

We also found that the robot sometimes **lost the target when it got too close**. This could interrupt the grabbing process. To improve this, we added target locking, stage switching thresholds, and lost-target recovery logic. These changes made the robot more stable during close-range operations.

In the competition environment, another issue was that **the robot could move too close to the marker-ball area** and make unnecessary or risky actions there. To solve this, we introduced forbidden-zone logic, which prevented the robot from selecting targets in that area while searching for items.

On the hardware side, the main issue was the gripper. **The original gripper was inadequate for grasping paper-pack drinks**, causing failures even when software performed correctly. To solve this, we redesigned the gripper and 3D printed two versions. The final version significantly improved grip reliability.

5.5 Testing Procedures

To make sure the robot could work properly in the real competition environment, we carried out testing step by step instead of only doing final full-process testing.

First, we did **object detection testing** using the actual target drinks and marker balls in a realistic arena background. This helped us check whether the model could recognize the objects correctly under real lighting and viewing conditions.

Second, we performed **target-centering and approaching tests**. In this stage, we focused on whether the robot could move smoothly toward detected objects and

maintain alignment. This was mainly used to adjust PID parameters, motor speeds, and threshold settings.

Third, we carried out **grasping tests**, especially for paper-pack drinks, because they were harder to handle than cans. These tests helped us check whether the robot stopped at the correct distance, whether the servo timing was suitable, and whether the object could be held securely after pickup.

Fourth, we tested the **drop-zone search and release process**. After the robot grabbed an object, we checked whether it could find the correct marker ball, approach the drop area properly, and release the object successfully.

We also tried **multi-object and cluttered-scene tests**. These tests were important because the real competition field was not a simple single-target environment. We used them to check whether target locking, forbidden-zone logic, and recovery functions worked well when multiple objects appeared together.

Finally, we carried out **full end-to-end tests on the actual competition field**. These tests covered the complete task flow from recognition to release. After repeated adjustments, the system demonstrated a **high completion rate** and stable performance under competition conditions.

5.6 Strategies and Techniques for Learning to Learn (L2L)

This project involved AI, embedded control, hardware design, and system integration, so we need to learn in an iterative and practical way instead of only relying on theory.

One important strategy we used was **breaking the project into smaller parts**. We did not try to solve everything at once. Instead, we divided the work into dataset preparation, object detection, motion control, gripper design, and final integration. This made the learning process more manageable.

Another strategy was **learning through repeated testing and improvement**. Many problems could not be solved by reading notes or documentation only. We often had to test the robot, observe what went wrong, and then adjust the code or hardware step by step. This helped us understand the real behavior of the system better.

We also used **reflection based on actual results**. Instead of changing things randomly, we tried to look at the problem carefully and think about whether it came from the model, the control logic, the hardware structure, or the environment. This made our learning process more efficient.

In addition, we learned a lot through **team discussion and explanation**. Since different members focused on different parts, face-to-face discussion helped us share what we learned and understand the project more completely. Through this process, we not only improved the robot, but also improved our ability to learn across different technical areas.

5.7 Team Collaboration

Team collaboration played an important role in this project because the system involved software, model training, hardware design, and testing at the same time.

Simon mainly worked on the **model and hardware side**, including dataset-related work and gripper design. Jiawei also mainly contributed to the **model and hardware side** and also helped with documentation-related tasks. ChingKwong was mainly responsible for the **software part**, including robot control logic, software integration, and debugging.

Although each member had a main role, we did not work completely separately. We mainly used **face-to-face communication** during development, testing, and debugging. This allowed us to discuss problems quickly and make changes efficiently.

For example, when the robot could not reliably grasp paper-pack drinks, we did not treat it as only a hardware problem. Instead, we discussed it together and considered both software timing and gripper design. Similarly, when the robot behaved unstably in the arena, we also checked the model, software, and hardware together instead of blaming only one part.

Overall, our teamwork was based on clear division of work, frequent communication, and repeated joint debugging. This helped us combine model performance, controlling logic, and hardware design into one complete working system.

6. Results and Evaluation

6.1 Outcomes of the Project

The project successfully delivered a fully functional autonomous retrieval robot capable of completing both Game 1 and Game 2 scenarios. During the final Game Day demonstration, the system performed as follows:

- Correctly discriminated between target drinks (Coca-Cola, sprite, Vita Lemon Tea, Vitasoy Soybean Milk) and distractor items (Coca-Cola black).
- Navigated to pick-up zones, executed reliable gripping using the custom 3D-printed gripper, transported items to the appropriate drop zones identified by green or red balls, and released items without knocking over adjacent objects.
- Completed both games within nine minutes, achieving full marks.

Key achieved application features and their impact are summarized in Table 2.

Table 2: Achieved Features and Impact

Feature	Implementation	Impact
Real-time multi-class detection	YOLOv11 TensorRT model, 416×416 input, 23-42 FPS	Enabled reliable target identification during navigation
Forbidden mask mechanism	Ground-plane projection with 15-frame memory	Prevented invalid pickup attempts behind or near balls
Blind-drop logic	Width-based threshold (drop_threshold=180 pixels)	Compensated for near-field camera blind spot
Custom gripper	8 cm 3D-printed gripper with anti-slip pads	Extended capability to both cans and paper-pack drinks
Chassis friction mitigation	Standoffs and axle offset	Improved straight-line tracking and turn reliability

6.2 Evaluation of Success Against Objectives

The project met all stated objectives:

1. **Hardware assembly and integration:** The robot platform was fully assembled, with Jetson Orin, camera, motors, and servo all operational.

2. **Object detection model development:** A custom dataset of 1,652 images and 2,564 annotations was curated and used to train a YOLOv11 model optimised for edge deployment.
3. **Autonomous retrieval:** The robot demonstrated consistent ability to locate, approach, grasp, transport, and release target objects without human intervention.
4. **Mechanical customisation:** The redesigned gripper and chassis modifications directly addressed limitations of the stock kit and were critical to Game 2 completion.

The investment in a large dataset and iterative hardware refinement proved decisive in achieving repeatable, high-confidence performance on Game Day.

6.3 Areas for Improvement

- **Far-field detection:** Detection confidence decreased for objects beyond approximately 1.2 metres. Adding more long-distance training images would improve far-field sensitivity.
- **Sprite class consistency:** The sprite class had the smallest annotation count (241) among active classes and showed the highest misclassification rate. Dataset expansion would improve parity.
- **Blind-drop precision:** The open-loop timed release introduced minor positional variability. Integrating a low-cost distance sensor would enable closed-loop approach control.
- **Gripper angular tolerance:** The custom gripper required precise approach alignment for paper-pack drinks. A wider jaw opening or passive compliance mechanism would increase robustness.

7. Conclusion

Summary of Key Contributions

This project successfully delivered an autonomous retrieval robot using NVIDIA Jetson Orin. Key contributions included:

1. A custom dataset of 1,652 images (2,564 annotations) for YOLOv11 training.
2. TensorRT-optimised detection at 23-42 FPS on embedded hardware.
3. A 3D-printed 8 cm gripper enabling dual-format gripping (cans and paper-packs).
4. Software adaptations: PID tracking, forbidden masks, and blind-drop logic.
5. Chassis modifications (standoffs, axle offset) improving navigation reliability.

Success and Limitations

The project met all objectives, achieving full marks on Game Day. Methodical dataset development and hardware iteration were critical to this outcome. Limitations included monocular vision blind spots and dataset bias toward mid-range imagery—both mitigated through software, though future work would benefit from sensor fusion.

Future Enhancements

With existing hardware and software, the following are achievable:

- Add distance sensors (e.g., ultrasonic) to reduce camera blind spot issues.
- Expand dataset with more varied angles, backgrounds, and distances.
- Upgrade wheels and bearings for smoother turning and straight-line movement.
- If hardware allows, use higher resolution input and a better camera for improved detection.
- Refine and optimize the code structure for better maintainability and efficiency.

The project provided valuable hands-on experience in end-to-end robotics development, from mechanical design to embedded AI deployment.

8. References and Appendix

Division of Workload Among Team Members

Group member	Responsibilities	Workload (%)
Li Ming Chun Simon (25017659D)	- Methodology and Description (Supplementary to hardware/software) - Implementation (Step-by-step details, Model parameters, Robot mechanics) - Results and Evaluation - Conclusion (Findings, Limitations, Future work)	33%
ZHONG Jiawei (21098707D)	- Introduction - Background and Objectives - Methodology and Description (Hardware/Software components, Object detection process, Integration)	33%
Hon Ching Kwong (25021497D)	- Methodology and Description (Supplementary details) - Milestone Achievement - Implementation (Additional features, Challenges & Solutions, Testing, L2L, Team collaboration)	34%
Total		100%